

Devoir surveillé terminal

Durée 2h00
(Corrections)

Exercice 1 (Cellula - 10 pts)

Nous souhaitons réaliser une simulation de type jeu de la vie nommée *Cellula*©. Le principe consiste à faire vivre une colonie de cellules dans un environnement donné, que nous modélisons par une grille de taille variable (elle peut contenir des obstacles). Régulièrement, chaque colonie sélectionne une de ses cellules et la fait évoluer en observant son environnement (les 4 cases autour d'elle). Si une de ces cases contient une cellule d'une autre colonie, elle l'absorbe (la case est donc occupée par une nouvelle cellule de la colonie). Sinon, s'il y a de l'espace autour d'elle (une case vide), la cellule se divise sur une des cases choisie aléatoirement.

Chaque colonie de cellules est simulée par une application différente. Elle s'arrête si elle a perdu toutes ses cellules. Il est possible d'en lancer au maximum N . L'application principale, nommée *Pétri*, prend en arguments la valeur de N , la largeur et la hauteur de la grille. Elle crée et initialise les outils IPC nécessaires.

File de messages

Une file de messages IPC est mise en place pour communiquer entre l'application *Pétri* et les colonies de cellules. Lorsqu'une colonie démarre, elle envoie un message à l'application *Pétri* qui lui donne l'autorisation ou non de démarrer. Lorsque l'application *Pétri* s'arrête, elle envoie un message aux colonies pour qu'elles s'arrêtent et attend une réponse de chacune d'elles, avant de s'arrêter à son tour. Enfin, si une colonie s'arrête (via une action de l'utilisateur ou si elle ne possède plus de cellules), elle envoie un message à l'application *Pétri* pour la prévenir.

1°) Y'a-t-il des problèmes de synchronisation lors de l'écriture dans une file de messages ?

Solution : (0,5 pt) La lecture et l'écriture sont gérées par le système. Il n'y a donc pas de problèmes de synchronisation lors de lectures/écritures concurrentes.

2°) Quelles sont les données échangées dans la file de messages (n'oubliez pas de préciser les types) ?

Solution : (2 pts) Ici, comme toutes les applications lisent et écrivent en même temps dans la file, il est nécessaire d'identifier de manière unique chaque application et de spécifier le destinataire des messages via le type. On peut utiliser le type 1 pour envoyer un message à l'application *Pétri* et le type X pour chaque colonie avec X le PID du processus de la colonie. Chaque message possède un code pour pouvoir être interprété correctement.

- Colonie $\rightarrow$ Pétri, **connexion** : type = 1 (long), code = 1 (unsigned char), PID (pid_t)
- Pétri $\rightarrow$ Colonie, **réponse connexion** : type = PID (long), code = 1 pour OK, 2 pour KO
- Colonie $\rightarrow$ Pétri, **déconnexion** : type = 1 (long), code = 2 (unsigned char), PID (pid_t) (ce message est envoyé lors d'une déconnexion ou bien à la suite d'une demande de déconnexion)
- Pétri $\rightarrow$ Colonie, **demande d'arrêt** : type = PID (long), code = 2 (unsigned char)

3°) Donnez la structure du message de connexion envoyé par la colonie, puis donnez le code permettant d'envoyer ce message (la clé de la file est fixée par la constante CLE). Déclarez et initialisez chaque variable nécessaire.

Solution : (2 pts) Voici la structure et le code :

```
typedef struct {
    long type;
    unsigned char code;
    pid_t pid;
} connexion_t;

int msgid = msgget(CLE, 0);
connexion_t msg;
msg.type = 1;
msg.code = 1;
msg.pid = getpid();
msgsnd(msgid, &msg, sizeof(msg), 0);
```

Segment de mémoire partagé

Le segment de mémoire partagé permet de partager les données entre l'application *Pétri* et les colonies de cellules. Il contient la grille de taille variable et ses dimensions, et le nombre de cellules de chaque colonie (ce nombre est mis-à-jour par la colonie ou les autres colonies).

4°) Proposez une structure permettant de mapper le segment.

Solution : (0,5 pt) Par exemple :

```
typedef struct {
    int shmid;
    void *adresse;
    int *largeur;
    int *hauteur;
    unsigned char *cases;
    int *nb_cellules;
} segment_t;
```

5°) Écrivez la fonction permettant de mapper le segment (en supposant qu'il existe). Sa signature est la suivante (la clé du segment est fixée dans la constante CLE) :

segment_t* mapper()

Solution : (2 pts) Par exemple :

```
segment_t mapper() {
    segment_t *retour = malloc(sizeof(segment_t));

    retour->shmid = shmget(CLE, 0, 0);
    retour->adresse = shmat(retour->shmid, 0, NULL);
    retour->largeur = (int*)retour->adresse;
    retour->hauteur = &retour->largeur[1];
    retour->cases = (unsigned char*)&retour->hauteur[1];
    retour->nb_cellules = (int*)retour->cases[retour->largeur * retour->hauteur];

    return retour;
}
```

Sémaphores

6°) Proposez une solution pour gérer les problèmes d'accès concurrents au segment de mémoire partagé, tout en permettant autant que possible une exécution parallèle des différentes applications.

Solution : (1 pt) Chaque case doit être accédée en exclusion mutuelle donc un sémaphore par case (initialisé à 1). Lorsqu'on accède à une case, on doit faire un $P(S)$ où S est le sémaphore de la case et une fois l'opération

terminée, on doit faire un $V(S)$. Un sémaphore est nécessaire pour le nombre de cellules de chaque colonie. On peut utiliser un seul sémaphore ou 1 pour chaque colonie. Il(s) est/sont initialisé(s) à 1 également.

7°) Supposons qu'une colonie désire faire évoluer la cellule située sur la case (x, y) . Expliquez comment réaliser cela tout en gérant les problèmes de synchronisation.

Solution : (1 pt) Il est nécessaire de vérifier les cases autour. Le plus simple : avec un sémaphore par case, on bloque les 5 cases en une seule fois. Si la case contient toujours une cellule, la colonie peut la faire évoluer (ajout sur une case à côté). Il faut ensuite mettre à jour les compteurs de cellules : celui de la colonie et celle qui a potentiellement perdu une cellule.

8°) Comment les colonies peuvent-elle détecter leur fin ? À quels moments doivent-elles faire cette vérification ?

Solution : (1 pt) On imagine que la colonie s'endort pendant un certain temps. À son réveil, elle doit vérifier qu'elle possède encore des cellules dans le segment. Si ce n'est pas le cas, le programme s'arrête. Sinon, elle doit choisir une de ses cellules sur toute la grille. Dès qu'une cellule est ciblée, il faut bloquer sa case et les 4 cases voisines. Dès que c'est le cas, on vérifie que la cellule est toujours là (on ne fait pas évoluer une cellule qui n'existe plus) et si ce n'est pas le cas, est-ce qu'il reste encore une cellule dans la colonie (peut-être qu'il s'agissait de la dernière cellule).

Exercice 2 (Cheapster - 10 pts)

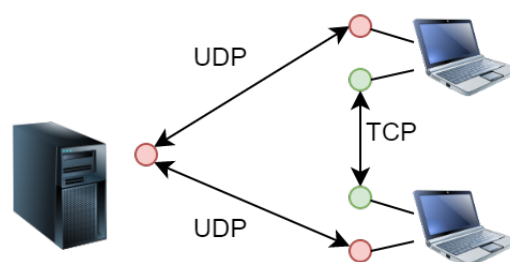
Nous souhaitons développer une application de partage de fichiers en mode pair-à-pair nommée *Cheapster*©. Les fichiers sont découpés en blocs de taille fixe (1024o) et partagés entre les clients. Un serveur attend des requêtes UDP de la part de clients. Ceux-ci peuvent envoyer un enregistrement de fichier en envoyant simplement son nom. Le serveur conserve la liste de tous les fichiers et les adresses IP des clients associés. Les clients peuvent également envoyer une demande de fichier. Dans ce cas, le serveur envoie une liste aléatoire d'adresses IP de taille fixe parmi celles associées au fichier. Lorsqu'un client la reçoit, il peut établir une connexion TCP vers les clients pour leur demander la liste des blocs qu'ils possèdent, puis leur demander de télécharger des blocs spécifiques. Les numéros de port sont fixés dans l'application et identiques sur tous les acteurs.

1°) À votre avis, est-ce que le choix d'UDP pour les requêtes entre le serveur et les clients est appropriée ? Comment peut-on gérer la perte de données ?

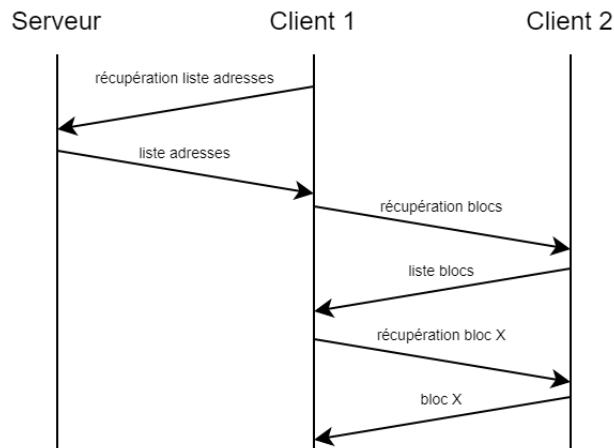
Solution : (1 pt) Il s'agit de simples envois de données, sans dialogue complexe et sans échange de données de grande taille. Donc UDP est suffisant ici. Le seul problème peut être la perte de données qui peut être gérée au niveau de l'application avec un simple compte-à-rebours. Par contre, cela nécessite que chaque échange soit acquitté (ce qui n'est pas le cas pour l'enregistrement d'un fichier).

2°) Illustrez l'architecture de l'application sous la forme d'un schéma (le serveur, 2 clients), en précisant les types de communication et les sockets.

Solution : (1 pt)



3°) Donnez un diagramme d'échange (un chronogramme) permettant d'illustrer les échanges en partant de la récupération de la liste des clients jusqu'à la récupération d'un bloc du fichier depuis un autre client. Précisez les données échangées (ainsi que leur type).

Solution : (2 pts)

Les données :

- *Récupération liste adresses* : nom du fichier (`char[255]`)
- *Liste adresses* : nombre d'adresses (`int`), tableau d'adresses (`sockaddr_in[10]`)
 ↪ En cas d'erreur, le nombre d'adresses est 0
- *Récupération blocs* : nom du fichier (`char[255]`)
- *Liste blocs* : nombre de blocs (`int`), numéros de blocs (`nombre de blocs × int`)
 ↪ En cas d'erreur, le nombre d'adresses est 0
- *Récupération bloc X* : nom du fichier (`char[255]`), numéro du bloc (`int`)
- *Bloc X* : données (`char[1024]`) et taille (`size_t`)
 ↪ En cas d'erreur, la taille est 0

4°) Expliquez comment un client peut récupérer simultanément 2 blocs d'un même fichier depuis 2 clients différents. Combien de sockets sont nécessaires ?

Solution : (1 pt) Le plus simple est de créer un fils pour chaque bloc. Le fils établit une connexion vers le client, récupère le bloc (voire éventuellement d'autres), puis s'arrête. Il faut donc une socket pour chaque fils, donc une pour chaque bloc. À noter qu'une connexion pourrait être utilisée pour récupérer plusieurs blocs.

5°) Comment le serveur peut-il récupérer l'adresse IP des clients lors de l'enregistrement d'un fichier ? Donnez les appels systèmes nécessaires.

Solution : (1 pt) Lorsqu'un message est récupéré avec `msgrcv`, il est possible de récupérer l'adresse dans le troisième paramètre (tout en spécifiant la taille de l'adresse dans le quatrième). L'adresse récupérée (`struct sockaddr_in` pour IPv4) peut être stockée sans conversion pour simplifier la suite.

Nous supposons que le client exécute la fonction `echange` dans un fils donné. Elle crée la socket qui permet d'attendre la connexion des autres clients. Chaque client qui se connecte est géré dans un fils spécifique (le code correspond à la fonction `fils`).

6°) Donnez le code de la fonction `echange` (le numéro de port TCP est fixé dans la constante `PORT_TCP`). Précisez les paramètres nécessaires pour la fonction `fils`.

Solution : (3 pts) Il s'agit d'un serveur TCP classique avec création de fils lors de chaque connexion (cf CM sur les sockets).

```

int fd, sockclient, stop = 0;
struct sockaddr_in adresse;

fd = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);

memset(&adresse, 0, sizeof(struct sockaddr_in));
  
```

```
adresse.sin_family = AF_INET;
adresse.sin_addr.s_addr = htonl(INADDR_ANY);
adresse.sin_port = htons(PORT_TCP);
bind(fd, (struct sockaddr*)&adresse, sizeof(struct sockaddr_in));
listen(fd, 1);

while(stop == 0) {
    sockclient = accept(fd, NULL, NULL);
    if(fork() == 0) {
        close(fd);

        fils(sockclient);
    }
    else
        close(sockclient);
}
close(fd);
```

7°) Un fichier étant récupéré bloc par bloc, dans un ordre aléatoire, expliquez comment stocker les données (données du fichier et numéros des blocs récupérés) dans un/des fichier(s).

Solution : (1 pt) *Le plus simple est de créer deux fichiers : le fichier à proprement parler (fichier de données) et un fichier permettant de stocker simplement un tableau de booléens (fichier d'état). Lorsqu'un bloc est récupéré, on se place à la position souhaitée dans le fichier de données et on stocke le bloc. Puis on va dans le fichier d'état et on met le booléen du bloc à vrai.*